

Compilateurs vérifiés : CompCert et CakeML

Programmation Comparée

Gabin Dudillieu Fares Mestre

Université Paris Cité

Janvier 2026

Introduction

Motivation

Bug GCC

```
int main(void) {  
    for (unsigned a = 0, b = 0; a < 6; a += 1, b += 2)  
        if (b < a)  
            return 1;  
    return 0;  
}
```

Bug GCC

```
int main(void) {  
    for (unsigned a = 0, b = 0; a < 6; a += 1, b += 2)  
        if (b < a)  
            return 1;  
    return 0;  
}  
  
$ gcc -Wall -Wextra unsigned-bug.c; ./a.out; echo $?  
0
```

Bug GCC

```
int main(void) {  
    for (unsigned a = 0, b = 0; a < 6; a += 1, b += 2)  
        if (b < a)  
            return 1;  
    return 0;  
}  
  
$ gcc -Wall -Wextra unsigned-bug.c; ./a.out; echo $?  
0  
$ gcc -Wall -Wextra unsigned-bug.c -O1; ./a.out; echo $?  
1  
$ gcc -Wall -Wextra unsigned-bug.c -O2; ./a.out; echo $?  
1  
$ gcc -Wall -Wextra unsigned-bug.c -O3; ./a.out; echo $?  
1
```

Bug GCC

```
int main(void) {  
    for (unsigned a = 0, b = 0; a < 6; a += 1, b += 2)  
        if (b < a)  
            return 1;  
    return 0;  
}  
  
$ gcc -Wall -Wextra unsigned-bug.c; ./a.out; echo $?  
0  
$ gcc -Wall -Wextra unsigned-bug.c -O1; ./a.out; echo $?  
1  
$ gcc -Wall -Wextra unsigned-bug.c -O2; ./a.out; echo $?  
1  
$ gcc -Wall -Wextra unsigned-bug.c -O3; ./a.out; echo $?  
1
```

Potentiellement introduit lors d'un commit d'optimisation sur GCC en 2018, et en 2021 touchait toutes les versions entre 8.3 et 9.1 de GCC.

"Fruit flies like a banana"

Est-ce que c'est le fruit qui vole comme une banane, ou la drosophile qui aime la banane ? (extrait du livre "Semantics of Programming Languages : Structures and Techniques" de Carl A. Gunter)

Compilateurs vérifiés

- Vérifiées mathématiquement
- Garantissent la propagation des propriétés de sécurité du langage source jusqu'au langage compilé
- Préservent la sémantique lors des phases de transition et d'optimisation

$$\forall S, B, C \notin Wrong,$$
$$Comp(S) = OK(C) \wedge S \Downarrow B \Rightarrow C \Downarrow B$$

- Prouver que le code d'une passe de compilation respecte la spécification
- Vérifier le résultat de la transformation

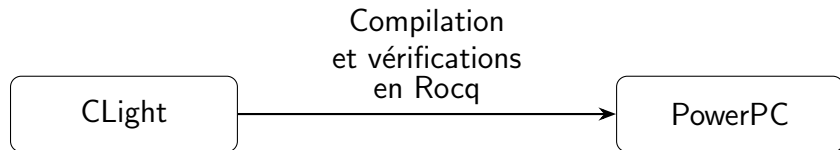
Vérification d'un compilateur



Focus sur CompCert

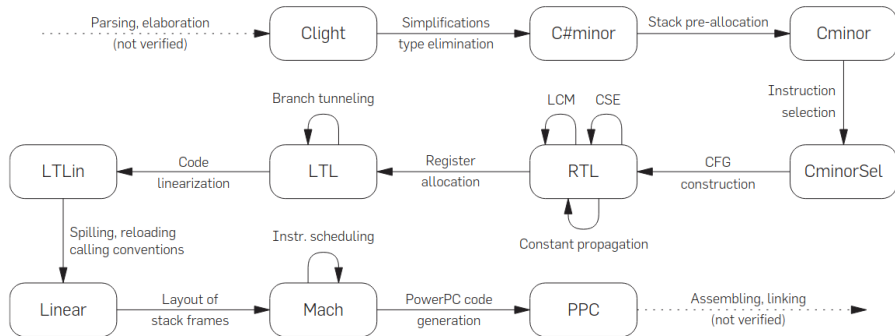
- Développé par Xavier Leroy, Sandrine Blazy et l'équipe Cambium de l'INRIA
- Conception de logiciels critiques
- Associé avec AbsInt, un outil d'analyse statique et de vérification formelle





- Large ensemble du langage C
- Utilisé dans la conception de logiciels critiques
- Supporte presque tous les concepts fondamentaux de C (majorité des types de données et des structures de contrôles, les fonctions récursives, les pointeurs de fonctions, ...)
- Sémantique déterministe
- Précision de comportements d'erreurs ou indéterminés

Compilation



Afficher un diagramme de simulation

- Chaque transition d'un programme source correspond à une séquence de programmes dans le langage transformé
- Cette séquence doit générer les mêmes effets observables
- Elle préserve comme invariant une relation binaire $\sim$ entre les états d'exécutions

Exemple avec RTL

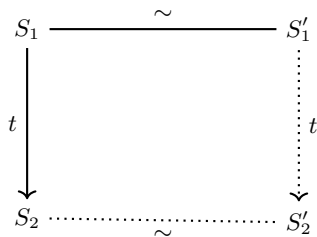
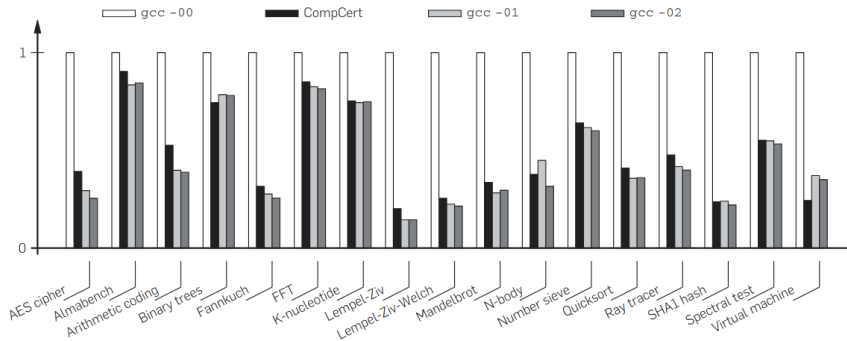


Diagramme de simulation de RTL

Définir relation $\sim$ avec S_1 et S_2 de la forme $S(\Sigma, g, s, l, R, M)$

Performances



Focus sur CakeML

- Langage de programmation fonctionnelle
- Dérivé de la spécification ML
- Conçu spécifiquement pour être compilé par un compilateur vérifié

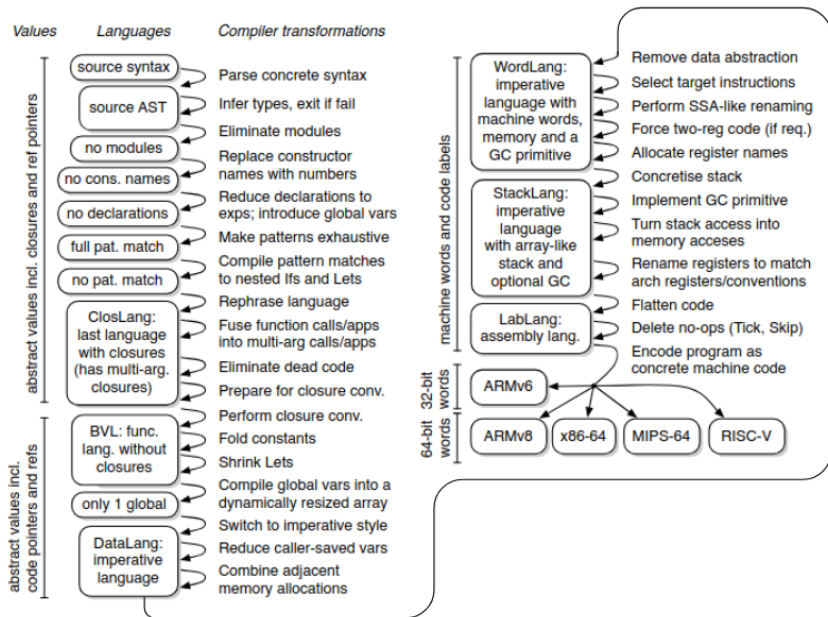


- Style curryfié (~~List.Map Some~~ $\vdash \rightarrow$ List.map (fun x $\Rightarrow$ Some x) l)
- Pattern matching qui ne prend pas d'exceptions
- Pas de foncteurs, d'enregistrements, de signature et d'open

- Evaluation de droite à gauche
- Egalité traitée différemment :
 - On a $(\text{fun plus2 } n = n + 2) = (\text{fun plus3 } n = n + 3)$
 - Mais $(\text{let val } x = [] \text{ in } (1::x, "str"::x)) \rightarrow \text{Erreur de typage}$

- Implémenté avec l'assistant de preuve HOL4 → Isabelle
- Peut se compiler lui-même (bootstrapé)
- Extensible
- Réutilisé dans la conception d'autres compilateurs vérifiés (PureCake, Kalas, Pancake, ...)

Compilation



- Sémantique à deux niveaux dans le style fonctionnel "big-step"
 - Sémantique d'évaluation "evaluate" : interpréteur du langage où les fonctions retournent le résultat de l'exécution
 - Sémantique d'observation "semantics" : les fonctions retournent un ensemble de comportements observables (terminaisons, divergence, ...)
- Les fonctions ne divergent pas (horloge pour "evaluate")

Spécification de la compatibilité des résultats du code source et du code compilé

compile $prog = new_prog \Rightarrow$

Fail $\notin \mathbf{semantics}_A prog \Rightarrow$

$\mathbf{semantics}_B new_prog = \mathbf{semantics}_A prog$

Spécification plus faible

$\mathbf{semantics}_B new_prog \subseteq$

$\mathbf{extend_with_resource_limit}(\mathbf{semantics}_A prog)$

Différentes approches de la compilation vérifiée

CompCert

- Compilation d'un sous-ensemble de C vers de l'assembleur en utilisant Rocq
- Beaucoup d'optimisations
- Utilisable dans un contexte industriel
- Limité à C

CakeML

- Compilation d'un sous-ensemble de ML en utilisant Isabelle
- Plus expressif
- Preuves plus complètes
- Preuves formelles complètes du langage
- Plus académique

Conclusion

Bibliographie (I)

- [1] Xavier Leroy, *Formal Verification of a Realistic Compiler*, 2009
- [2] Sandrine Blazy and Xavier Leroy, Mechanized semantics for the Clight subset of the C language, 2009
- [3] Yong Kiam Ta et al., *A New Verified Compiler Backend for CakeML*, 2016
- [4] ACM, *PureCake: A Verified Compiler for a Lazy Functional Language*, 2023.
- [5] Mattox Beckman, *Big Step Semantics*, 2020
- [6] ACM, *Finding and Understanding Bugs in C Compilers*, 2011
- [7] Carl A. Gunter, *Semantics of Programming Languages : Structures and Techniques*, 1992

Bibliographie (II)

- [1] GNU.org, *Bug C*, https://gcc.gnu.org/bugzilla/show_bug.cgi?id=102666#c0
- [2] CompCert.org, *CompCert*, <https://compcert.org/>
- [3] CakeML.org, *CakeML*, <https://cakeml.org/>
- [4] Github, *CompCert*, <https://github.com/AbsInt/CompCert/>
- [5] Github, *CakeML*, <https://github.com/CakeML/cakeml/>
- [6] Wikipédia, *CompCert*, <https://fr.wikipedia.org/wiki/CompCert>
- [7] Wikipédia, *Hindley-Milner type system*,
https://en.wikipedia.org/wiki/Hindley%E2%80%93Milner_type_system
- [8] Wikipédia, *Sémantique des langages de programmation*,
https://fr.wikipedia.org/wiki/S%C3%A9mantique_des_langages_de_programmation